

Prob 1. (7 pts) Assume we call an ASM function to return a signed 32-bit value. If the value is -2, determine: (1) (2 pts) the register used for returning the value and (2) (5 pts) the 32-bit value in the return register in hexadecimal.

Soln:

- (a) R0.
- (b) R0 = 0xFFFF_FFFE.

∴

Prob 2. (20 pts) Suppose we want to design two test cases for a 64-bit addition function, such as the one given below

```
1 mp_add_64bit_s:
2     adds    r0, r2
3     adc     r1, r3
4     bx      lr
```

Assume we use the following two sets of numbers

```
1 int64_t A1 = (int64_t)0x377<<24, B1 = (int64_t)0x287<<24;
2 int64_t A2 = (int64_t)0x378<<24, B2 = (int64_t)0x288<<24;
```

1. (14 pts) Determine the values of $A1 + B1$ and $A2 + B2$ and write them in C code format like the ones given above.
2. (6 pts) Determine the values of the C flag in the above two operation, respectively.

Part 1.

- $A1 + B1 = (int64_t)0x5FE<<24$
- $A2 + B2 = (int64_t)0x600<<24$

Part 2.

- $A1 + B1: C = 0$
- $A2 + B2: C = 1$

∴

Prob 3. (20 pts, 5 each) Consider the problem of saving register values for the caller inside a function. Assume we have the following given conditions:

- Register R_i has a value of $(1 \ll i) + (1 \ll (2 + i))$, where $i = 1, 2, \dots, 12$. For example, the value of R3 is $(1 \ll 3) + (1 \ll 5) = 0x28$.
- The value of SP is 0x2000_1020 before pushing new numbers.
- We use descending full stack, as discussed in the class notes.
- We use `push {r4, r8, r6}` to push values of three registers on the stack.

Answer the following questions:

1. What is the value of SP after pushing the above registers?
2. What is the value of R4 in 32-bit hexadecimal?
3. What is the value of the **word** at address 0x2000_1018?
4. What is the value of the **byte** at address of 0x2000_1019?

Part 1. After pushing each register, we have the value of SP decremented by 4. Hence, after pushing 3 registers, we have SP's value decremented by 12. Hence, the value of SP is 0x2000_1014.

Part 2. The value of R4 is $(1 \ll 4) + (1 \ll 6) = 0x0000_0050$.

Part 3. R6 is saved at 0x2000_1018. Hence, the value in word at this address is $(1 \ll 6) + (1 \ll 8) = 0x0000_0140$.

Part 4. The value of the byte at 0x2000_1019 is a byte of R6, and it is 0x01 from the above calculation.

∴-

Prob 4. (25 pts total, 5 pts each) We have learned in the class that using the barrel shifter, we can implement multiplication/division of a number for some special cases with addition, subtraction, and reversed subtraction instructions as well as shift instructions. For each of the following calculations, write a **single-line assembly code** (based on Op2) to perform it using this approach assuming **signed** integers **A** and **B** are saved in **r0** and **r1**, respectively:

- 1 (a) $A = 6 * B / 8$
- 2 (b) $A = 7 * B / 8$
- 3 (c) $A = 8 * B / 8$
- 4 (d) $A = 9 * B / 8$
- 5 (e) $A = 10 * B / 8$

Soln:

- 1 (a) $A = 6 * B / 8$: `sub r0, r1, r1, asr #2`
- 2 (b) $A = 7 * B / 8$: `sub r0, r1, r1, asr #3`
- 3 (c) $A = 8 * B / 8$: `mov r0, r1 or add r0, r1, #0`
- 4 (d) $A = 9 * B / 8$: `add r0, r1, r1, asr #3`
- 5 (e) $A = 10 * B / 8$: `add r0, r1, r1, asr #2`

∴-

Prob 5. (20 pts) Assume we have the following assembly function

```
1 @ uint16_t func_s(uint16_t a, uint16_t b);
2 func_s: PROC
3     ldr        r2, =15
4     bic        r0, r2, LSL #8
5     and        r1, r2
6     orr        r0, r1, LSL #8
7     bx        lr
```

- (1) (12 pts). Translate the above ASM function to a C function based on a defined variable named `mask` with the following prototype.

```
1 uint16_t func_c(uint16_t a, uint16_t b);
```

(2) (8 pts). If $a = 0xH_3H_2H_1H_0$, what is the value of a after running

```
1 a = func_c(a, 17);
```

Solution:

Part 1:

```
1 uint16_t func_c(uint16_t a, uint16_t b) {
2     uint16_t mask = 15;
3     a &= ~(mask << 8);
4     a |= (b & mask) << 8;
5     return a;
6 }
```

Part 2: $a = 0xH_31H_2H_0$.

:-

Prob 6. (8 pts) Suppose we have defined two variables as `int8_t var1= 0x7E;` and `int8_t var2= 0x81;` and run the following statement: `fn(var1, var2);`. Determine the values of registers R0 and R1 in 32-bit hexadecimal immediately after the program runs within function `void fn(int32_t x, int32_t y)` due to this call in hexadecimal format.

R0 = 0x0000_007E and R1 = 0xFFFF_FF81.

:-